BACHELORARBEIT · VERSTÄNDLICH ERKLÄRT · PRÜFRECHNUNGEN

Wie prüft die Arbeit?

Dieses Heft setzt „Wie rechnen die Modelle?“ fort und zählt dessen Teile weiter. Teil 10 zeigt, wie aus 50 Läufen die Ergebnistabellen und Tests werden, Teil 11 rechnet weitere Rechnungen aus dem Text nach. Die Teile 12 bis 15 erklären, was die Arbeit an Daten und Verfahren prüft und misst: die Überdispersion, die Tests der Eignungsprüfung, die Korrelationsmaße mit der Intraklassenkorrelation und den Rechenaufwand.

Titel der Arbeit: „Vorhersage von Feuerwehreinsätzen auf Basis struktureller und sozioökonomischer Merkmale – ein Vergleich ausgewählter Machine-Learning-Methoden am Beispiel von Stadtteildaten“
Grundlage: abgegebene Fassung vom 07.10.2026 und der Abgabe-Code (Ordner Bachelorarbeit_LukasNiessen_[redacted])
Zahlen: Teile 10 und 11 am 11.10.2026 nachgerechnet, Teile 12 bis 14 am 12.10.2026 aus den Daten des Abgabeordners (sie treffen die Werte der Ergebnisdateien), Teil 15 nach Tabelle 13 der Arbeit
Seitenangaben: „S.“ meint immer die Seite in der Arbeit
Markierung: ZUSATZ kennzeichnet Angaben, die nicht in der Arbeit stehen
Stand: 12.10.2026

TEIL 10

Vom Einzellauf zum Ergebnis

> Ein Fold ergibt einen RMSE. Die Arbeit rechnet aber 50 Läufe je Verfahren: zehn Wiederholungen mit je fünf Folds. Dieser Teil zeigt, wie daraus die Zahlen in Tabelle 9 und Tabelle 10 der Arbeit werden (S. 60–61).

Zweistufig mitteln

Gemittelt wird in zwei Schritten: erst über die fünf Folds einer Wiederholung, dann über die zehn Wiederholungsmittel. Die Streuung hinter jedem Mittelwert ist die dieser zehn Werte und nicht die der 50 einzelnen Folds (S. 41–42). Für Ridge auf der Einsatzzahl sind das diese zehn Wiederholungsmittel:

29,30; 30,05; 29,35; 31,57; 28,53; 31,75; 30,49; 33,25; 32,19; 32,77

Ihr Mittel ist 30,93, ihre Standardabweichung 1,61. Das ist der Eintrag 30,93 ± 1,61 in Tabelle 9 der Arbeit (S. 60). Die Streuung über die 50 einzelnen Folds wäre mit 21,40 viel größer (menge_mittel.csv), weil sie auch enthält, dass manche Folds leicht und andere schwer sind. Die Arbeit nennt für die drei Verfahren 19,92 bis 23,81 (S. 59).

```
448  schluessel = ["zielgroesse", "verfahren"]
449  g = folds.groupby(schluessel, sort=False)
450  z = g[MASSE].mean().add_suffix("_mean")
451  z = z.join(g[MASSE].std().add_suffix("_std_folds"))
452  je_wdh = folds.groupby(schluessel + ["wiederholung"], sort=False)[MASSE].mean()
453  z = z.join(je_wdh.groupby(schluessel, sort=False).std()
454                   .add_suffix("_std_wiederholungen"))
```

modelle/m02_menge.py, Z. 448–454

Z. 450 Der Mittelwert direkt über alle 50 Läufe. Bei fünf Folds je Wiederholung ist er derselbe wie zweistufig gemittelt (S. 42).

Z. 452–453 Erst das Mittel je Wiederholung, dann die Standardabweichung über diese zehn Werte: die Streuung, die die Arbeit berichtet.

Gepaart vergleichen: Ridge gegen XGBoost

Weil beide Verfahren in jeder Wiederholung dieselben Testfolds sehen, wird je Wiederholung die Differenz gebildet (S. 59). Positiv heißt: Ridge hat den kleineren Fehler.

Wiederholung	0	1	2	3	4	5	6	7	8	9	Mittel
XGBoost	33,59	33,30	33,80	44,47	39,98	32,73	34,81	37,85	43,97	39,41	37,39
Ridge	29,30	30,05	29,35	31,57	28,53	31,75	30,49	33,25	32,19	32,77	30,93
Differenz Δ_w	**+4,29**	**+3,24**	**+4,45**	**+12,90**	**+11,45**	**+0,98**	**+4,32**	**+4,60**	**+11,77**	**+6,64**	**+6,46**

Tabelle 1: Die zehn Wiederholungsmittel des RMSE und ihre Differenz, aus `results/regression/menge_folds.csv`.

Aus den zehn Differenzen entstehen die drei Kennzahlen, die Tabelle 10 der Arbeit (S. 61) neben jeden p-Wert stellt: die mittlere Differenz, ihr 95-%-Konfidenzintervall nach der t-Verteilung und die Effektstärke d, also die mittlere Differenz geteilt durch die Standardabweichung der Differenzen (S. 19).

$$\bar{\Delta} = \frac{1}{10}\sum_{w=1}^{10}\Delta_w \qquad d = \frac{\bar{\Delta}}{s_\Delta} \qquad KI = \bar{\Delta} \pm t_{0{,}975;\,9}\cdot\frac{s_\Delta}{\sqrt{10}}$$

Mittlere Differenz, Effektstärke und Konfidenzintervall · in der Arbeit in Worten (S. 19)

1	Mittlere Differenz	Mittel der zehn Δ_w	**+6,46**
2	Standardabweichung	der zehn Δ_w	**s_Δ = 4,107**
3	Effektstärke	6,465 / 4,107	**d = 1,57**
4	Halbe Breite	2,262 · 4,107 / √10	**2,94**
5	Konfidenzintervall	6,465 ± 2,938	**[+3,53; +9,40]**

Das sind genau die Werte der Zeile „Ridge gegen XGBoost“ in Tabelle 10 der Arbeit: +6,46 [+3,53; +9,40], d = 1,57. Im Code entsteht das Intervall in `_gepaart()`, die Effektstärke rechnet `trennschaerfe.py` in Z. 44–53 aus dem Intervall zurück. Das Ergebnis ist dasselbe.

Der Wilcoxon-Test

Der Vorzeichen-Rang-Test nach Wilcoxon ordnet die Beträge der zehn Differenzen nach ihrer Größe und summiert die Ränge der positiven (S. 11). Hier sind alle zehn Differenzen positiv, alle Ränge fallen also auf die positive Seite. Wie selten das ohne echten Unterschied wäre, lässt sich direkt abzählen ZUSATZ: Gäbe es keinen Unterschied, wäre jedes Vorzeichen wie ein Münzwurf. Von den 2^{10} = 1.024 gleich wahrscheinlichen Vorzeichenmustern ist nur eines so extrem, zweiseitig zählt das umgekehrte mit. Also ist p = 2/1.024 = 0,00195.

Mit gemischten Vorzeichen rechnet der Test über die Rangsummen. Das Heft „Frage 10 und 16 einfach erklärt“ zeigt das am Vergleich Ridge gegen Poisson-GLM: Rangsumme der positiven 39, der negativen 16, p = 0,275.

Die Korrektur nach Holm

Die Vergleiche der Verfahren untereinander bilden im Mengenstrang eine Familie aus sechs Tests: drei Paarungen auf zwei Zielgrößen (S. 19). Holm prüft ihre p-Werte der Größe nach: den kleinsten gegen die strengste Schwelle, jeden weiteren gegen eine etwas mildere, bis der erste scheitert (S. 12). Gleichwertig lässt sich jeder p-Wert mit einem Faktor multiplizieren: den kleinsten mit sechs, den nächsten mit fünf und so weiter. Als angepasster p-Wert geschrieben, der direkt mit 0,05 verglichen wird, gilt dabei jeweils das bisherige Maximum, damit kein späterer Wert unter einem früheren landet ZUSATZ.

$$p_{(i)}^{\text{Holm}} = \max_{j\le i}\ \min\Big(1,\ (m-j+1)\cdot p_{(j)}\Big)$$

Korrektur nach Holm als angepasster p-Wert, m Tests, p-Werte aufsteigend sortiert · Zusatz, das Verfahren steht auf S. 12

Rang	Vergleich	p	Faktor	p · Faktor	p nach Holm	Datei
1	Ridge – XGBoost, Anzahl	0,00195	6	0,0117	**0,0117**	0,0117
2	Ridge – XGBoost, Rate	0,00195	5	0,0098	**0,0117**	0,0117
3	Ridge – Random Forest, Anzahl	0,00586	4	0,0234	**0,0234**	0,0234
4	Ridge – Random Forest, Rate	0,01953	3	0,0586	**0,0586**	0,0586
5	Random Forest – XGBoost, Anzahl	0,10547	2	0,2109	**0,2109**	0,2109
6	Random Forest – XGBoost, Rate	0,43164	1	0,4316	**0,4316**	0,4316

Tabelle 2: Die Holm-Korrektur der sechs Vergleiche zwischen den Verfahren. Die vorletzte Spalte ist von Hand gerechnet, die letzte stammt aus `results/regression/vergleich.csv`. In Rang 2 gilt das Maximum aus Rang 1.

Gerundet sind das die Holm-Werte 0,012, 0,023 und 0,211 aus Tabelle 10 der Arbeit und die 0,059 auf der Rate (S. 61). Signifikant ist ein Vergleich, wenn sein Holm-Wert unter $\alpha = 0{,}05$ liegt: Das gilt für die Ränge 1 bis 3. Die Vergleiche mit der Baseline werden nicht korrigiert, weil jeder vorab einzeln festgelegt war (S. 19).

```
494  m = len(p)
495  ordnung = np.argsort(p)
496  angepasst = np.empty(m, float)
497  laufend = 0.0
498  for rang, i in enumerate(ordnung):
499      laufend = max(laufend, (m - rang) * p[i])
500      angepasst[i] = min(laufend, 1.0)
501  return angepasst
```

modelle/m02_menge.py, Z. 494–501

Z. 495 `argsort` sortiert die p-Werte aufsteigend.

Z. 499 `rang` beginnt im Code bei 0. Deshalb ist `m - rang` der Faktor aus Tabelle 2 (6, 5, …), `(m - rang) * p[i]` die Spalte p · Faktor und `max(laufend, …)` das bisherige Maximum.

Z. 500 Kein Wert darf über eins steigen.

TEIL 11

Weitere Rechnungen der Arbeit

> Drei Rechnungen im Text und eine Formel gehören nicht zu den Modellen, tauchen aber in der Arbeit auf. Sie sind hier knapp nachgerechnet.

Formel 7 · der Kriminalitätsindex

$$\mathrm{Rate}(s,m) = \frac{D(s,m)}{B(s,m)}, \qquad \mathrm{Index}(s,m) = \frac{\mathrm{Rate}(s,m)}{\mathrm{Rate}(\mathrm{Stadt},m)}$$

Formel 7 der Arbeit · Kriminalitätsindex als Standortquotient (S. 34)

- $D(s,m)$ — Die Delikte im Stadtteil s in den zwölf Monaten bis einschließlich m − 1. Der laufende Monat zählt nicht mit (S. 34).
- $B(s,m)$ — Die Wohnbevölkerung des Stadtteils im zugeordneten Zensusjahrgang.
- $\mathrm{Rate}(\mathrm{Stadt},m)$ — Dieselbe Rate über alle Stadtteile zusammen: alle Delikte durch alle Einwohner (S. 34).

Ein Rechenbeispiel mit runden, erfundenen Zahlen ZUSATZ: Ein Stadtteil hat 900 Delikte in den zwölf Vormonaten und 25.000 Einwohner, also 36 Delikte je 1.000 Einwohner. Die ganze Stadt hat 51.000 Delikte bei 850.000 Einwohnern, also 60 je 1.000. Der Index ist 36/60 = 0,60. Der Stadtteil ist damit um 40 % weniger belastet als der Stadtdurchschnitt desselben Monats, ein Wert von eins hieße genau durchschnittlich (S. 35). Ins Modell geht der Logarithmus ein, hier log(0,60) = −0,51.

Echt: Bernal Heights hat im Oktober 2019 den Index 0,617, im Modell steht log(0,617) = −0,4829 (Tabelle 2 im Heft „Wie rechnen die Modelle?“). Im Code bildet `prep/s1_daten.py` in Z. 510 das Zwölfmonatsfenster ohne den laufenden Monat, Z. 522–524 die Stadtsummen und Z. 526–530 beide Raten und ihren Quotienten. Beide Raten sind dort je 1.000 Einwohner gerechnet, der Faktor kürzt sich im Index heraus. Den Logarithmus nimmt `prep/s2_datensaetze.py` in Z. 244.

Designeffekt und effektive Stichprobe

Die 4.752 Zeilen sind keine unabhängigen Beobachtungen: Jeder Stadtteil steht mit 132 einander sehr ähnlichen Monaten darin. Aus einer Intraklassenkorrelation von 0,926 folgt (S. 41):

$$1 + (132 - 1) \cdot 0{,}926 = 122{,}3 \qquad \frac{4.752}{122{,}3} = 38{,}9$$

Designeffekt 1 + (Monate je Stadtteil − 1) · Intraklassenkorrelation und effektive Stichprobe (S. 41)

Die Zeilen tragen also ungefähr so viel Information wie 39 unabhängige Beobachtungen.

Das Suchbudget

Macht ein Bereich guter Hyperparameter den Anteil v/V des Suchraums aus, trifft eine Zufallssuche mit T Ziehungen ihn mit der Wahrscheinlichkeit $1 - (1 - v/V)^T$ mindestens einmal (S. 56). Für einen Bereich von 5 %:

$$1 - (1 - v/V)^T : \qquad 1 - 0{,}95^{100} = 0{,}994 \qquad 1 - 0{,}95^{50} = 0{,}923$$

Trefferwahrscheinlichkeit der Zufallssuche (S. 56) · Code: config_modelle.py, Z. 18–21

Daher die 100 Ziehungen je Suchlauf. Die Rechnung hängt nicht davon ab, wie viele Hyperparameter ein Verfahren hat (S. 56).

Zehn Tests auf einmal

Wer zehn unabhängige Tests zum Niveau 0,05 rechnet, findet mit dieser Wahrscheinlichkeit mindestens einen Unterschied, der gar nicht besteht (S. 12):

$$1 - 0{,}95^{10} = 0{,}401$$

Wahrscheinlichkeit mindestens eines Zufallsbefunds bei zehn Tests (S. 12)

Gegen genau diese Gefahr richtet sich die Holm-Korrektur aus Teil 10.

TEIL 12

Passt die Poisson-Annahme? Varianz und Überdispersion

> Das Poisson-GLM unterstellt, dass die Einsatzzahlen genau so stark streuen, wie ihr Erwartungswert groß ist. Dieser Teil zeigt, was das heißt, wie weit die Daten davon abweichen und warum das die Messlatte trotzdem nicht beschädigt.

Erwartungswert gleich Varianz

$$\mu = \mathbb{E}[Y] = \mathrm{Var}(Y)$$

Kennzeichen der Poisson-Verteilung (S. 5)

Bei einer Poisson-Verteilung ist die Varianz so groß wie der Erwartungswert: Je mehr Einsätze man erwartet, desto stärker schwanken sie (S. 5). Die typische Schwankung ist die Wurzel daraus. Erwartet man in einem Monat 4 Einsätze, liegt sie bei $\sqrt{4} = 2$, bei 100 erwarteten Einsätzen bei $\sqrt{100} = 10$ ZUSATZ. Ein lineares Modell mit konstanter Fehlervarianz kann diesen Zusammenhang nicht abbilden (S. 5).

Echte Zähldaten halten diese Gleichheit selten ein. Liegt die Varianz deutlich darüber, spricht man von Überdispersion (S. 5). Die Arbeit misst sie auf zwei Wegen: grob mit dem Dispersionsindex und formal mit einer Hilfsregression (S. 48, Tabelle 7, S. 49).

Der Dispersionsindex

$$\text{Index} = \frac{s^2}{\bar{y}}$$

Dispersionsindex · Zusatz, in Worten und mit dem Wert 79,5 auf S. 52 und in Tabelle 7 (S. 49)

Der Index teilt die Varianz s^2 der Einsatzzahlen durch ihren Mittelwert $\bar{y}$. Bei einer Poisson-Verteilung wären beide gleich, der Index also eins (S. 52). Gerechnet wird auf den Trainingszeilen des ersten Folds (S. 52):

FOLD 1, MITTELWERT UND VARIANZ ZUSATZ

1	Mittelwert	Einsatzzahl der 3.168 Trainingszeilen von Fold 1	**69,86**
2	Varianz	Streuung derselben Zeilen	**5.555,0**
3	Index	5.555,0 / 69,86	**79,5**

Die Varianz ist also fast 80-mal so groß, wie Poisson erwarten lässt. Der Index misst die Streuung um den Mittelwert aller Stadtteile, nicht um die Vorhersage des Modells (S. 48). Der weitaus größte Teil kommt deshalb daher, dass große und kleine Stadtteile in einem Topf liegen: Misst man nur die Schwankung der Monate innerhalb ihres Stadtteils, liegt der Index bei etwa 5 ZUSATZ. Für Random Forest und XGBoost folgt daraus die Verlustfunktion: Random Forest minimiert die Poisson-Devianz, XGBoost eine Tweedie-Devianz, statt des quadratischen Fehlers (S. 52, Tabelle 7, S. 49).

```
71  y = train[ZIELGROESSE].astype(float)
72  index = y.var() / y.mean()
```

vorpruefung/v2_eignung.py, Z. 71–72

Z. 72 `y.var() / y.mean()`: Varianz durch Mittelwert der Trainingszeilen, also genau die Rechnung oben.

Der Test nach Cameron und Trivedi

Der formale Test prüft die Streuung um die Vorhersage des Poisson-GLM. Er ist als Hilfsregression nach Cameron und Trivedi gebaut und der einzige der vier Tests, den keine Bibliothek mitbringt (S. 48).

$$z_i = \frac{(y_i - \hat{\mu}_i)^2 - y_i}{\hat{\mu}_i} = \alpha\,\hat{\mu}_i + \varepsilon_i$$

Hilfsregression nach Cameron und Trivedi · in der Arbeit in Worten (S. 48), Formel als Zusatz

DIE TEILE DER FORMEL

y_i Die wahre Einsatzzahl der Trainingszeile i.

$\hat{\mu}_i$ Die Vorhersage des Poisson-GLM mit Offset für diese Zeile. Geprüft wird also genau die spätere Messlatte (S. 49).

$(y_i - \hat{\mu}_i)^2$ Die beobachtete quadrierte Abweichung, eine grobe Schätzung der Varianz dieser Zeile ZUSATZ.

$-\,y_i$ Zieht den wahren Wert ab. Er steht für die Poisson-Varianz μ, denn im Mittel ist y = μ ZUSATZ. Ohne Überdispersion ist die Hilfsgröße z deshalb im Mittel null, mit Überdispersion wächst sie (S. 48).

α Der Koeffizient einer Regression von z auf μ̂ ohne Konstante. Er entspricht dem Dispersionsparameter der negativen Binomialverteilung, die Nullhypothese ist α = 0 (S. 48).

Warum gerade α · μ̂ ZUSATZ: Die negative Binomialverteilung erlaubt die Varianz μ + α μ². Zieht man μ ab und teilt durch μ, bleibt α μ übrig, also genau die rechte Seite. Eine einzelne Zeile, erfunden:

BEISPIEL ZUSATZ

1	Abweichung quadrieren	Vorhersage 50, wahr 70: (70 − 50)²	**400**
2	Wahren Wert abziehen	400 − 70	**330**
3	Durch die Vorhersage teilen	330 / 50	**z = 6,6**

Unter Poisson läge z im Mittel bei null. In den 3.168 Trainingszeilen von Fold 1 liegt es im Mittel bei 4,45 ZUSATZ. Die Regression fasst das in einer Zahl zusammen:

FOLD 1

1	Koeffizient	Regression von z auf μ̂ ohne Konstante ZUSATZ	**α̂ = 0,0375**
2	t-Wert	α̂ durch seinen Standardfehler: 0,0375 / 0,00208 ZUSATZ	**t = 18,0**
3	p-Wert	einseitig, der p-Wert wird halbiert (S. 48)	**< 0,001**

Was α̂ = 0,0375 bedeutet ZUSATZ: Bei 70 erwarteten Einsätzen wäre die Varianz 70 + 0,0375 · 70² = 253,8 statt 70, die typische Schwankung also 15,9 statt 8,4 Einsätze. Die Annahme ist verletzt (Tabelle 7, S. 49). Folgen hat das für die Messlatte nicht: Ist der Erwartungswert richtig angesetzt, bleibt der Poisson-Schätzer konsistent, verzerrt werden nur die Standardfehler (S. 5). Diese berichtet die Arbeit nicht, die Messlatte liefert nur Punktvorhersagen (S. 48, Tabelle 7, S. 49). Ob der Erwartungswert selbst richtig angesetzt ist, prüft der RESET-Test in Teil 13 ZUSATZ.

```
403  X = sm.add_constant(train[MERKMALE].astype(float), has_constant="add")
404  y = train[ZIELGROESSE].astype(float).to_numpy()
410  poisson = sm.GLM(y, X, family=sm.families.Poisson(),
411                   offset=np.log(train[EXPOSURE_ROH].astype(float))).fit()
412  mu = np.asarray(poisson.fittedvalues, float)
413  z = ((y - mu) ** 2 - y) / mu
414  hilfs = sm.OLS(z, mu).fit()
415  ct_t, ct_p = float(hilfs.tvalues[0]), float(hilfs.pvalues[0]) / 2
```

vorpruefung/v2_eignung.py, Z. 403–415, gekürzt (Code 14 der Arbeit)

Z. 410–411 Dasselbe Poisson-GLM mit demselben Offset wie die Messlatte (S. 49).

Z. 413 Die Hilfsgröße z aus der Formel oben.

Z. 414 `sm.OLS(z, mu)` ohne `add_constant`: die Regression ohne Konstante, ihr Koeffizient ist $\hat{\alpha}$.

Z. 415 Der p-Wert wird halbiert. Das macht den Test einseitig, denn eine Varianz unter dem Erwartungswert wäre bei Einsatzzahlen keine sinnvolle Gegenhypothese (S. 48).

TEIL 13

Die Eignungsprüfung: Tests und ein Maß

> Bevor die Verfahren antreten, prüft die Arbeit, ob die Daten ihre Anforderungen erfüllen (S. 18). Gerechnet wird auf den 24 Trainingsstadtteilen des ersten Folds, bis auf den VIF. Weil die Monate eines Stadtteils nicht unabhängig sind, gelten alle p-Werte nur nominell (S. 18).

Prüfung	Frage	Wert	Folge
Hilfsregression (Teil 12)	Varianz gleich Erwartungswert?	t = 18,0	folgenlos, nur Punktvorhersagen
RESET	Reicht eine lineare Form?	F = 360,4	Trennlinie zu den Baumverfahren
Breusch-Pagan	Streuen die Residuen überall gleich?	LM = 525,1	betrifft Standardfehler, nicht die Prognose
Jarque-Bera	Sind die Residuen normalverteilt?	JB = 150,0	Voraussetzung der Inferenz, nicht der Prognose
VIF	Wie stark hängen die Merkmale zusammen?	10,64	Ridge statt Kleinste Quadrate (S. 46)
Kruskal-Wallis	Trennen die Merkmale die Einsatzarten?	H = 30,6 bis 459,6	Strukturstrang findet statt (S. 47)

Tabelle 3: Die Prüfungen der Eignungsprüfung im Überblick (Tabelle 7, S. 49, sowie S. 46–47). Die Tests sind auf den Trainingsstadtteilen des ersten Folds gerechnet, nominell sämtlich mit $p < 0{,}001$, der VIF auf den 30 Entwicklungsstadtteilen ZUSATZ.

Der RESET-Test rechnet auf log(1 + y), also auf logarithmischer Skala wie Ridge und die Referenz (S. 45). Breusch-Pagan und Jarque-Bera prüfen im Code dasselbe Modell, eine gewöhnliche lineare Regression auf log(1 + Anzahl) mit den zwölf Merkmalen ZUSATZ. Tabelle 7 ordnet ihre Werte Ridge zu (S. 49). Gemeinsam ist die logarithmische Skala, Ridge selbst schätzt die Rate (S. 51–52) ZUSATZ.

RESET: Fehlt dem Modell eine Krümmung?

Der Test passt das lineare Modell an und nimmt dessen quadrierte Vorhersagen als zusätzliches Merkmal auf. Ist die lineare Form richtig, gewinnt das Modell dadurch nichts. Wird es besser, fehlt ihm etwas, das eine Gerade nicht abbilden kann (S. 45). Gemessen wird der Gewinn mit einem F-Wert ZUSATZ:

$$F = \frac{(RSS_{ohne} - RSS_{mit}) / 1}{RSS_{mit} / (n - k)}$$

F-Wert des RESET-Tests mit einem Zusatzterm ŷ² · Zusatz

RSS ist die Summe der quadrierten Fehler, einmal ohne und einmal mit ŷ². Der Zähler ist der Gewinn je Zusatzterm, der Nenner der verbleibende Fehler je Freiheitsgrad. n − k sind die Zeilen minus die geschätzten Koeffizienten: 3.168 − 14 = 3.154 ZUSATZ.

FOLD 1, ZWISCHENWERTE ZUSATZ

1	Fehlerquadrate ohne ŷ²	lineares Modell auf log(1 + y)	**345,10**
2	Fehlerquadrate mit ŷ²	ein Zusatzterm	**309,71**
3	Gewinn	345,10 − 309,71	**35,39**
4	Fehler je Freiheitsgrad	309,71 / 3.154	**0,0982**
5	F-Wert	35,39 / 0,0982	**360,4**

Ein einziger Zusatzterm senkt die Fehlerquadrate um gut 10 %, ohne fehlende Struktur läge F in der Nähe von eins ZUSATZ. Hier sind es 360,4. Was fehlt, sind Krümmung und Wechselwirkungen: Nimmt man alle 45 Produkte aus je zwei Strukturmerkmalen auf, steigt das adjustierte Bestimmtheitsmaß von 0,859 auf 0,932 (S. 46). Die linearen Verfahren scheiden trotzdem nicht aus. Der Test misst auf denselben Daten, an denen angepasst wurde, und entscheiden muss der Vergleich (S. 46).

```
202  y = np.log1p(train[ZIELGROESSE].astype(float))
203  ols = sm.OLS(y, sm.add_constant(train[MERKMALE].astype(float),
204                                  has_constant="add")).fit()
209  for potenz in (2, 3):
210      r = linear_reset(ols, power=potenz, test_type="fitted", use_f=True)
211      reset[potenz] = (float(r.fvalue), float(r.pvalue))
```

vorpruefung/v2_eignung.py, Z. 202–211, gekürzt

Z. 202 `np.log1p`: die Zielgröße als log(1 + y).

Z. 210 `linear_reset` mit `test_type="fitted"` nimmt Potenzen der Vorhersage auf, `use_f=True` liefert den F-Wert. Die Schleife rechnet Potenzen bis 2, das ist der Wert der Arbeit, und bis 3 ZUSATZ.

Breusch-Pagan: Streuen die Residuen überall gleich?

Der Test fragt, ob die Residuen in allen Bereichen der Daten gleich stark streuen. Dazu regressiert er die quadrierten Residuen auf dieselben Merkmale. Erklären die Merkmale einen Teil dieser Streuung, ist sie ungleich ZUSATZ. Die ursprüngliche Form setzt normalverteilte Störgrößen voraus, genau das verwirft der Jarque-Bera-Test. Gerechnet ist deshalb die Voreinstellung von statsmodels, die keine Normalverteilung voraussetzt (S. 50).

$$\mathrm{LM} = n \cdot R^2_{\mathrm{Hilf}}$$

Prüfgröße in der Voreinstellung von statsmodels · Zusatz

FOLD 1, ZWISCHENWERTE ZUSATZ

1	Hilfsregression	quadrierte Residuen auf die zwölf Merkmale	**R^2 = 0,16574**
2	Prüfgröße	3.168 · 0,16574	**LM = 525,1**
3	Schwelle zum Vergleich	χ^2-Verteilung mit 12 Freiheitsgraden, 5 %	**21,0**

Die Merkmale erklären 16,6 % der Streuung der quadrierten Residuen, bei gleicher Streuung wären es nahe null ZUSATZ. Die Verletzung betrifft die Standardfehler, nicht die Prognose (Tabelle 7, S. 49).

Jarque-Bera: Sind die Residuen normalverteilt?

$$\mathrm{JB} = \frac{n}{6}\left(S^2 + \frac{(K-3)^2}{4}\right)$$

Jarque-Bera-Statistik · Zusatz, Schiefe und Wölbung stehen auf S. 50

Der Test vergleicht zwei Formmerkmale der Residuen mit der Normalverteilung. Die Schiefe S misst, ob die Verteilung zu einer Seite hängt, bei einer Normalverteilung ist sie null ZUSATZ. Die Wölbung K misst, wie viele extreme Werte vorkommen, bei einer Normalverteilung ist sie 3,0 (S. 50). Beide Abweichungen gehen quadriert ein ZUSATZ.

FOLD 1, UNGERUNDET: SCHIEFE 0,0218, WÖLBUNG 4,065 ZUSATZ

1	Schiefe quadriert	$0{,}0218^2$	**0,0005**
2	Wölbungsterm	$(4{,}065 - 3)^2 / 4$	**0,2836**
3	Prüfgröße	3.168 / 6 · (0,0005 + 0,2836)	**JB = 150,0**

Fast der ganze Wert kommt aus der Wölbung ZUSATZ: Die Residuen sind symmetrisch, haben aber mehr extreme Werte als eine Normalverteilung (S. 50). Für die Prognose ist Normalverteilung nicht erforderlich, sie ist eine Voraussetzung der Inferenz (Tabelle 7, S. 49).

```
422  for ziel in (ZIELGROESSE, RATE):
423      ols = sm.OLS(np.log1p(train[ziel].astype(float)), X).fit()
424      bp = het_breuschpagan(ols.resid, ols.model.exog)
425      jb = jarque_bera(ols.resid)
```

vorpruefung/v2_eignung.py, Z. 422–425

Z. 423 Die lineare Regression auf log(1 + y) mit den zwölf Merkmalen.

Z. 424–425 `het_breuschpagan` und `jarque_bera` auf ihren Residuen. Die Schleife rechnet Anzahl und Rate, die Werte der Arbeit sind die der Anzahl ZUSATZ.

VIF: Wie stark hängen die Merkmale zusammen?

Der Varianzinflationsfaktor gibt an, um welchen Faktor die übrigen Merkmale die Varianz eines Koeffizienten erhöhen. Ab 5 bis 10 gilt er als problematisch (S. 46).

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2}$$

Varianzinflationsfaktor des Merkmals j · Zusatz

R_j^2 ist das Bestimmtheitsmaß, wenn man das Merkmal j aus den übrigen neun Strukturmerkmalen vorhersagt. Lässt es sich gar nicht vorhersagen, ist R_j^2 null und der VIF eins. Je besser die anderen es vorhersagen, desto größer wird er ZUSATZ.

30 ENTWICKLUNGSSTADTTEILE, R² ZUSATZ

1	Hilfsregression	Medianeinkommen aus den übrigen neun Strukturmerkmalen	**R² = 0,906**
2	VIF	1 / (1 − 0,906)	**10,64**

Das Medianeinkommen lässt sich also zu 90,6 % aus den anderen Merkmalen vorhersagen ZUSATZ. Über alle Stadtteile ist seine Korrelation mit der Medianmiete die größte der ganzen Korrelationsmatrix, 0,921 (S. 27). Gerechnet ist der VIF auf den 30 Entwicklungsstadtteilen, maßgeblich auf einer Zeile je Stadtteil und Jahr, zum Vergleich auf allen Zeilen. Beide ergeben 10,64 (`results/shap/vif.csv`) ZUSATZ. Die Folge steht in der Arbeit: Statt der gewöhnlichen Kleinste-Quadrate-Schätzung tritt Ridge an, dessen Strafterm die Folgen dämpft, ohne ein Merkmal zu entfernen (S. 46).

```
443 for basis, teil in mengen.items():
444     X = add_constant(teil[PRAEDIKTOREN].astype(float), has_constant="add")
445     for i, s in enumerate(X.columns):
446         if s == "const":
447             continue
448         zeilen.append({"basis": basis, "merkmal": s, "n_zeilen": len(teil),
449                        "vif": round(float(variance_inflation_factor(
450                            X.to_numpy(), i)), 2)})
```

modelle/m04_shap.py, Z. 443–450

Z. 444 Die zehn Strukturmerkmale mit Konstante, ohne die Saisonterme.

Z. 449–450 `variance_inflation_factor` rechnet je Merkmal die Hilfsregression und 1 / (1 − R²).

Kruskal-Wallis: Trennen die Merkmale die Einsatzarten?

Für die Einsatzart klärt die Arbeit vorab, ob sich die Merkmale zwischen den vier Einsatzarten überhaupt unterscheiden. Der Kruskal-Wallis-Test ist die verteilungsfreie Verallgemeinerung des Wilcoxon-Rangsummen-Tests auf mehr als zwei Gruppen und setzt keine bestimmte Verteilung voraus (S. 47). Er rechnet nicht mit den Werten, sondern mit ihren Rängen ZUSATZ.

$$H = \frac{12}{N(N+1)} \sum_{k} n_k (\bar{R}_k - \bar{R})^2$$

Kruskal-Wallis-Statistik ohne Korrektur für gleiche Werte · Zusatz

Alle N Zeilen werden nach dem Merkmal sortiert und durchnummeriert, gleiche Werte teilen sich den mittleren Platz ZUSATZ. Im ersten Fold sind es 3.167 statt 3.168 Zeilen, weil ein Stadtteilmonat ohne Einsatz keine Einsatzart hat (S. 25). $\bar{R}_k$ ist der mittlere Rang der n_k Zeilen der Einsatzart k, $\bar{R} = (N + 1)/2$ der mittlere Rang aller Zeilen. Unterscheiden sich die Einsatzarten nicht, liegen alle $\bar{R}_k$ nahe bei $\bar{R}$ und H bleibt klein ZUSATZ.

LEERSTANDSQUOTE, FOLD 1 ZUSATZ

1	Mittlerer Rang aller Zeilen	(3.167 + 1) / 2	**1.584**
2	Mittlere Ränge je Einsatzart	Brand 1.151; Fehlalarm 1.761; Rettung 718; Technische Hilfe 952	
3	Gewichtete Abstände	z. B. Rettung 131 · (718 − 1.584)² = 98,2 Mio., alle vier zusammen	**384,15 Mio.**
4	Prüfgröße	12 / (3.167 · 3.168) · 384,15 Mio.	**H = 459,5**

Mit der Korrektur für gleiche Werte, die scipy anbringt, sind es 459,6 ZUSATZ, der höchste der zehn Werte (S. 47). Bei vier Gruppen liegt die Schwelle für 5 % bei 7,8 ZUSATZ. Alle zehn Strukturmerkmale liegen mit 30,6 bis 459,6 darüber, der Strukturstrang findet damit statt (S. 47). Dass sie die Klassen trennen, heißt, dass sie eine Klassenzuordnung zulassen, nicht dass eines von ihnen die Aussage trägt (S. 71).

```
300  for m in PRAEDIKTOREN:
301      gruppen = [train.loc[train[ZIELKLASSE] == k, m].astype(float)
302                 for k in klassen]
303      h, p = kruskal(*gruppen)
```

vorpruefung/v2_eignung.py, Z. 300–303

Z. 301–302 Je Merkmal vier Gruppen, eine je Einsatzart, aus den Trainingszeilen von Fold 1.

Z. 303 `kruskal` aus scipy liefert H und p.

TEIL 14

Zusammenhänge messen: Pearson, Spearman und die Intraklassenkorrelation

> Die Arbeit beschreibt ihre Daten mit drei Zusammenhangsmaßen. Pearson und Spearman messen, wie ein Merkmal mit der Einsatzzahl zusammenhängt. Die Intraklassenkorrelation misst, wie ähnlich sich die Monate eines Stadtteils sind.

Pearson und Spearman

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \, \sum_i (y_i - \bar{y})^2}} \qquad \rho_S = r \text{ der Ränge}$$

Korrelation nach Bravais-Pearson und Rangkorrelation nach Spearman · in Worten auf S. 26, Formel als Zusatz

Pearson misst den linearen Zusammenhang. Der Zähler wird groß, wenn Werte über dem Mittel bei x mit Werten über dem Mittel bei y zusammentreffen, der Nenner bringt das Ergebnis auf −1 bis +1 ZUSATZ. Spearman rechnet dieselbe Formel auf den Rängen statt auf den Werten ZUSATZ und misst damit den monotonen Zusammenhang (S. 26). Beträge unter 0,5 gelten als schwach, ab 0,5 als mittel, ab 0,8 als stark (S. 26).

Beispiel	x	y	Pearson	Spearman
gekrümmt	1; 2; 3; 4; 5	1; 2; 3; 4; 20	0,80	1,00
ein Punkt weit draußen	1; 2; 3; 4; 10	3; 1; 2; 4; 10	0,94	0,70

Tabelle 4: Zwei erfundene Reihen mit je fünf Punkten ZUSATZ.

Im ersten Beispiel steigt y immer mit x, aber nicht gleichmäßig. Die Ränge sind in beiden Spalten 1 bis 5, Spearman ist deshalb genau eins, Pearson nur 0,80. Liegt das Rangmaß höher, spricht das für einen monotonen, aber gekrümmten Verlauf (S. 26). Im zweiten Beispiel ordnen sich die ersten vier Punkte kaum, ein einzelner Punkt weit draußen zieht Pearson auf 0,94.

Genau diese beiden Muster zeigt die Arbeit. Bei der logarithmierten Wohnbevölkerung liegt Spearman höher: Pearson 0,480, Spearman 0,619 auf allen Zeilen (Tabelle 4, S. 27), 0,532 und 0,744 auf den Trainingszeilen des ersten Folds (S. 45). Bei der Armutsquote liegt Pearson vorn: 0,478 gegen 0,331 (Tabelle 4, S. 27), 0,617 gegen 0,342 im ersten Fold. Das deutet auf einzelne Stadtteile, die die Gerade an sich ziehen (S. 45).

```
393  for c in spalten:
394      p = d[c].corr(d[ziel], method="pearson")
395      s = d[c].corr(d[ziel], method="spearman")
```

prep/deskriptiv.py, Z. 393–395

Z. 394–395 `.corr(..., method=...)` aus pandas rechnet beide Maße je Merkmal gegen die Zielgröße.

Die Intraklassenkorrelation

Die Varianzzerlegung teilt die Streuung einer Größe in einen Anteil zwischen den Stadtteilen und einen innerhalb (S. 25). Die Intraklassenkorrelation gibt an, wie stark zwei Monate desselben Stadtteils miteinander korrelieren (S. 25). Der Code rechnet sie in der Einweg-Form ICC(1) ZUSATZ:

$$\mathrm{ICC} = \frac{\mathrm{MS}_{\mathrm{zw}} - \mathrm{MS}_{\mathrm{in}}}{\mathrm{MS}_{\mathrm{zw}} + (\mathrm{m} - 1)\,\mathrm{MS}_{\mathrm{in}}}$$

Intraklassenkorrelation ICC(1) · Zusatz, der Wert 0,926 steht auf S. 25

$\mathrm{MS}_{\mathrm{zw}}$ misst, wie weit die Stadtteilmittel auseinanderliegen, $\mathrm{MS}_{\mathrm{in}}$, wie stark die Monate um das Mittel ihres Stadtteils schwanken, beide je Freiheitsgrad. m ist die Zahl der Monate je Stadtteil, hier 132 ZUSATZ. Ein kleines Beispiel mit zwei Stadtteilen und je drei Monaten:

BEISPIEL ZUSATZ

1	Zwei Stadtteile	A: 10; 12; 14 (Mittel 12) · B: 50; 52; 54 (Mittel 52) · Gesamtmittel 32	
2	Streuung zwischen	3 · (20² + 20²) = 2.400, durch 1 Freiheitsgrad	**2.400**
3	Streuung innerhalb	(4 + 0 + 4) + (4 + 0 + 4) = 16, durch 4 Freiheitsgrade	**4**
4	ICC	(2.400 − 4) / (2.400 + 2 · 4)	**0,995**

Die Monate schwanken kaum um ihr Stadtteilmittel, fast die ganze Streuung liegt zwischen A und B. Ein weiterer Monat von A verrät deshalb kaum etwas Neues. In den echten Daten:

ALLE 36 STADTTEILE, ZWISCHENWERTE ZUSATZ

1	Streuung zwischen	36 Stadtteilmittel der Einsatzzahl, 4.752 Zeilen	**583.764,6**
2	Streuung innerhalb	Monate um ihr Stadtteilmittel	**353,5**
3	ICC	(583.764,6 − 353,5) / (583.764,6 + 131 · 353,5)	**0,926**

Dazu passt die Varianzzerlegung: 92,5 % der Varianz der Einsatzzahl liegen zwischen den Stadtteilen, 7,5 % innerhalb (S. 25). Beide Zahlen liegen nah beieinander, weil die Unterschiede zwischen den Stadtteilen die Schwankung der Monate weit übersteigen ZUSATZ. Aus der Intraklassenkorrelation folgt der Designeffekt aus Teil 11: Die 4.752 Zeilen tragen ungefähr so viel Information wie 39 unabhängige Beobachtungen (S. 41).

```
133  ss_zwischen = float((groesse * (mittel - gesamt) ** 2).sum())
134  ss_gesamt = float(((s - gesamt) ** 2).sum())
135  ss_innerhalb = ss_gesamt - ss_zwischen
136
137  ms_zwischen = ss_zwischen / (k - 1)
138  ms_innerhalb = ss_innerhalb / (n - k)
139  # ICC(1) nach Shrout & Fleiss; negative Werte auf 0 gekappt, weil eine
140  # negative Varianzkomponente keine Bedeutung hat.
141  nenner = ms_zwischen + (m - 1) * ms_innerhalb
142  icc = max(0.0, (ms_zwischen - ms_innerhalb) / nenner) if nenner else 0.0
143  deff = 1 + (m - 1) * icc
```

prep/deskriptiv.py, Z. 133–143

Z. 133–135 Die Quadratsumme zwischen den Stadtteilen, die gesamte und als Differenz die innerhalb.

Z. 137–138 Geteilt durch die Freiheitsgrade: 36 − 1 zwischen, 4.752 − 36 innerhalb.

Z. 142–143 ICC(1) wie in der Formel oben und daraus der Designeffekt aus Teil 11.

TEIL 15

Rechenaufwand: was O(·) bedeutet

> Ridge passt einen Fold in 0,008 Sekunden an, XGBoost braucht das 295-Fache, Random Forest das 326-Fache (S. 67). Die Arbeit erklärt den Abstand mit der Rechenlast je Anpassung und schreibt sie in O-Notation (S. 67).

	Ridge	XGBoost	Random Forest
Training, Einsatzhäufigkeit	0,008 s	2,40 s	2,66 s
Training, Rate	0,008 s	2,39 s	2,66 s
Training, Einsatzart	–	2,46 s	2,47 s
Inferenz, Einsatzhäufigkeit	0,003 s	0,021 s	0,078 s

Tabelle 5: Trainings- und Inferenzzeit je Fold, einkernig gemessen (Tabelle 13, S. 67). Die Faktoren 295 und 326 sind aus den ungerundeten Mittelwerten in `results/regression/menge_folds.csv` gerechnet ZUSATZ.

Das Landau-Symbol O(·) sagt, wie der Aufwand wächst, wenn die Daten größer werden. Feste Faktoren lässt es weg: O(n) heißt nur, dass doppelt so viele Zeilen ungefähr doppelt so viel Arbeit machen ZUSATZ. n ist die Zahl der Zeilen, q die Zahl der Merkmale.

Ridge: O(nq² + q³)

$$O(nq^2 + q^3)$$

Aufwand der Ridge-Anpassung (S. 67)

Ridge löst ein Gleichungssystem in den q Merkmalen (S. 67). Der erste Summand ist das Aufstellen von $X^\top X$ aus Formel 4a (S. 8): Für jedes der q · q Merkmalspaare wird über alle n Zeilen summiert. Der zweite ist das Lösen des Systems mit q Unbekannten ZUSATZ.

GRÖSSENORDNUNG FÜR FOLD 1 ZUSATZ

1	Aufstellen	$n\,q^2 = 3.168 \cdot 12^2$	**456.192**
2	Lösen	$q^3 = 12^3$	**1.728**
3	Zusammen	456.192 + 1.728	**457.920**

Bei nur zwölf Merkmalen bestimmt der erste Summand den Aufwand. Doppelt so viele Zeilen verdoppeln ihn, doppelt so viele Merkmale vervierfachen ihn ZUSATZ.

Ein Baum: O(q n log n)

$$O(q\,n\log n)$$

Aufwand eines einzelnen Entscheidungsbaums (S. 67)

Ein Baum sortiert die Beobachtungen je Merkmal (S. 67). Eine Liste mit n Werten zu sortieren kostet ungefähr n log n Schritte, und das für jedes der q Merkmale ZUSATZ.

GRÖSSENORDNUNG FÜR FOLD 1 ZUSATZ

1	Logarithmus	$\log_2 3.168$	**11,63**
2	Ein Baum	$12 \cdot 3.168 \cdot 11{,}63$	**≈ 442.000**
3	Random Forest in Fold 1	898 Bäume · 442.000	**≈ 397 Mio.**

Ein einzelner Baum kostet damit ungefähr so viel wie die ganze Ridge-Anpassung ZUSATZ. Den Abstand machen die vielen Bäume: Beide Ensembles vervielfachen diesen Posten mit der Zahl ihrer Bäume, und davon bauen sie je Fold mehrere hundert (S. 67), im ersten Fold sind es beim Random Forest 898 (Tabelle 8, S. 56–57). Mit ihnen ergibt die Überschlagsrechnung das 867-Fache, gemessen ist im Mittel über die Folds das 326-Fache. Die Rechnung trifft nur die Größenordnung, weil O(·) feste Faktoren und festen Grundaufwand weglässt und die Bäume je Fold verschieden viele sind ZUSATZ. XGBoost sortiert die Spalten einmal vorab und durchläuft danach je Baum und Ebene alle Einträge der Datenmatrix (S. 67), das sind n · q = 38.016 Einträge ZUSATZ.

Parallelisierung gleicht den Abstand nicht aus. Random Forest baut seine Bäume unabhängig voneinander und wird bei der Einsatzhäufigkeit um den Faktor 2,06 schneller, Ridge kaum (1,08). Bei XGBoost liegt der Wert mit 0,84 unter eins: Über alle vier logischen Kerne dauert die Anpassung länger als auf einem (S. 67).